

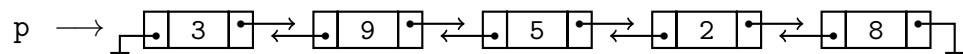
Estruturas de Dados

Lista de exercícios 09

1. O elemento central

Imagine que nós temos uma lista duplamente encadeada.

Por exemplo



A tarefa consiste em

→ *Imprimir o elemento central da lista, onde*

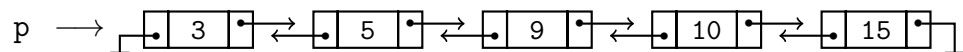
- se a lista tem tamanho ímpar, isso corresponde ao elemento do meio
- se a lista tem tamanho par, isso corresponde ao último elemento da primeira metade

- Apresente um algoritmo que resolve esse problema.
- Analise o tempo de execução do seu algoritmo.

2. Operação de atualização

Imagine que nós temos uma lista duplamente encadeada ordenada.

Por exemplo



A operação de atualização consiste em

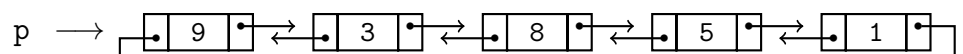
- localizar um elemento x na lista
- modificar o seu valor para y
- e depois movê-lo para a sua posição correta

- Apresente um algoritmo que implementa a operação de atualização.
- Analise o tempo de execução do seu algoritmo.

3. Varredura e ordenação

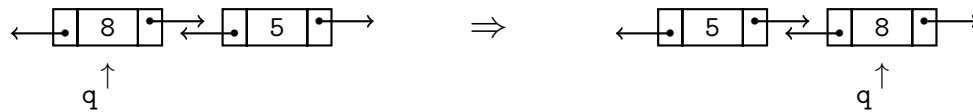
Imagine que nós temos uma lista duplamente encadeada não ordenada.

Por exemplo



A operação de *varredura* consiste em

- percorrer a lista da esquerda para a direita
- trocando o elemento atual de posição com o próximo, quando este é maior do que aquele



(Note que a varredura move o maior elemento para o final da lista)

(E note que varreduras sucessivas produzem uma lista completamente ordenada)

a) Apresente um algoritmo que implementa a operação de varredura.

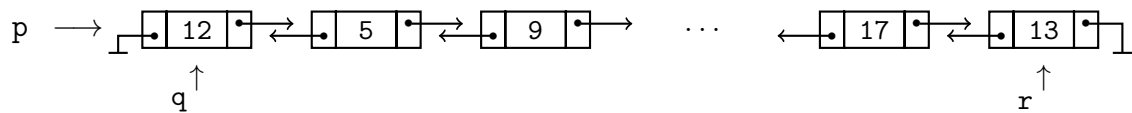
Nota: Você deve trocar os elementos de lugar na lista (quando necessário), e não apenas trocar os números armazenados neles.

Dica: Implemente a operação de troca como uma função em separado.

b) Analise o tempo de execução do seu algoritmo.

4. Partição

Imagine que nós temos uma lista duplamente encadeada (não ordenada), com ponteiros auxiliares q, r apontando para o primeiro e o último elementos



A tarefa consiste em, dado um número k

1. percorrer a lista da esquerda para a direita com o ponteiro q , até encontrar um elemento $> k$
2. percorrer a lista da direita para a esquerda com o ponteiro r , até encontrar um elemento $\leq k$
3. trocar os elementos apontados por q, r de posição
4. Repetir os passos 1 – 3 até que os ponteiros q, r se encontrem

(Note que esse procedimento move os elementos $\leq k$ para o início da lista, e move os elementos $> k$ para o final da lista)

a) Apresente um algoritmo que resolve esse problema.

b) Analise o tempo de execução do seu algoritmo.

5. Vetores esparsos

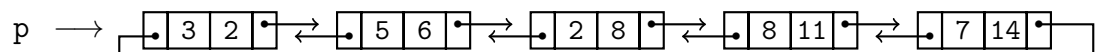
Imagine que nós temos um vetor onde a grande maioria dos elementos é 0 (**zero**).

Por exemplo

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
V	0	3	0	0	0	5	0	2	0	0	8	0	0	7	0

Esses vetores são conhecidos como *vetores esparsos*.

Nesse tipo de situação, faz sentido pensar em armazenar o vetor como um lista encadeada, da seguinte maneira



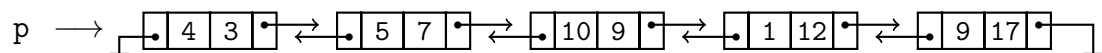
Quer dizer, cada elemento da lista armazena

- um valor diferente de zero
 - e uma indicação da posição no vetor onde esse valor está armazenado
- a) Apresente um algoritmo que constrói a representação de um vetor esparsos na forma de uma lista duplamente encadeada.
- b) Analise o tempo de execução do seu algoritmo.

6. Operações sobre vetores esparsos

Imagine que nós temos um vetor esparsos representado na forma de uma lista duplamente encadeada.

Por exemplo



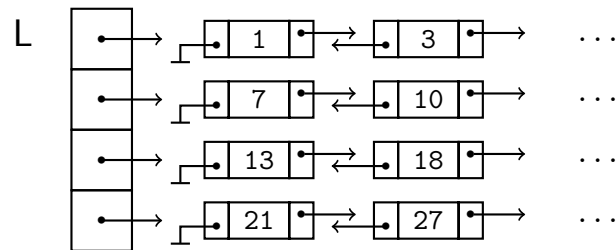
- a) Apresente algoritmos que implementam as seguintes operações sobre vetores esparsos
- a1) **Busca-por-índice(k)**: retorna o valor armazenado na posição k
 - a2) **Busca-por-valor(x)**: retorna a posição onde o valor x está armazenado
— (ou -1 se ele não está lá)
 - a3) **Atualização(x,k)**: atualiza o valor da posição k para x
- b) Analise os tempos de execução dos seus algoritmos.

9. Lista de listas

Uma *lista de listas* é um vetor de ponteiros, cada um deles apontando para uma sublista (duplamente) encadeada, onde

- as sublistas estão todas ordenadas
- os elementos de cada sublista são todos menores que os elementos da próxima sublista

Por exemplo



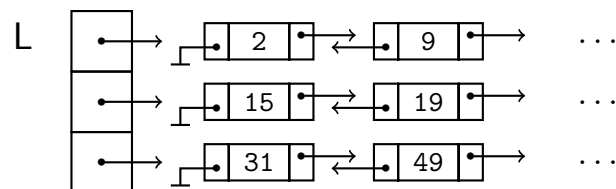
A vantagem dessa estrutura é que ela reduz o tempo de busca aos elementos — (*porque?*)

- Apresente um algoritmo que transforma uma lista duplamente encadeada em uma lista de listas de tamanho k — (*onde todas as sublistas tem (aproximadamente) o mesmo tamanho*)
- Analise o tempo de execução do seu algoritmo.

10. Operações na lista de listas

Imagine que nós temos uma lista de listas.

Por exemplo



- Apresente algoritmos que implementam as seguintes operações sobre a lista de listas
 - $\text{Busca}(L, x)$
 - $\text{Inserção}(L, x)$
 - $\text{Remoção}(L, x)$
- Analise os tempos de execução dos seus algoritmos.